

Migração para o servidor AWS — manual de execução

Aplicação WeeFly Concierge + Price Checker (Next.js 14)

De weefly.duckdns.org

Para concierge.weefly.africa — Elastic IP 52.30.78.0

Servidor EC2, fornecido pela equipa do Sarin Ram

Data 16 de Setembro de 2026

Para seguir por Fábio Rocha

Este documento é para ser seguido de cima a baixo, numa sessão. Cada parte acaba num estado verificável — se a verificação falhar, não se avança.

Não escrevas segredos neste documento. As chaves reais vêm do servidor antigo e do gestor de palavras-passe; este ficheiro circula por email.

Antes de começar

O que ainda falta pedir ao Sarin

Três destes são bloqueantes: sem eles não se passa da Parte 1.

O QUE FALTA	ESTADO	PORQUE É PRECISO
Utilizador SSH	Bloqueante	ubuntu, ec2-user ou admin, conforme a imagem. Sem isto não há sessão.
RAM da instância	Bloqueante	Decide se é preciso swap antes do primeiro build. Ver Parte 3.
Permissão de escrita em /var/www	Bloqueante	Define onde a aplicação vive e se é preciso sudo para cada deploy.
De quem é a conta AWS	Importante	Define se consegues mexer em security groups, redimensionar e tirar snapshots, ou se tens de pedir por email de cada vez. Muda o dia-a-dia durante meses.
Porta 3000 fechada no security group	Importante	Se estiver aberta, a aplicação responde sem passar pelo NGINX nem pelo SSL.

O QUE FALTA	ESTADO	PORQUE É PRECISO
Quem criou o registo DNS de <code>concierge.weefly.africa</code>	Importante	O DNS está na GoDaddy e nós não temos acesso. Se o certificado já foi emitido, alguém tem. É a mesma pessoa a quem voltarás a pedir registos.
Acesso sem porta 22	Recomendado	EC2 Instance Connect ou SSM Session Manager. É a saída para o dia em que o teu IP mudar.

O IP a dar-lhes para o whitelisting

TIPO	VALOR
IPv4	<code>85.246.221.246/32</code>
IPv6	<code>2001:8a0:574d:4900::/64</code>

Dá **os dois**. A linha tem IPv6 activo: se o SSH sair por IPv6 e a regra for só IPv4, a ligação é recusada sem explicação aparente.

E avisa-os de que o IPv4 é dinâmico. Quando a operadora o mudar, ficas fora e tens de mandar email a pedir. É por isso que a via alternativa da última linha da tabela acima importa mais do que parece.

Parte 1 - As chaves e o primeiro acesso

Tirar as chaves da pasta do repositório

As chaves estão em `E:\Projects\WeeFly\Weefly Backend\`. Não foram para o git — o `.gitignore` apanha `*.pem` e `*.ppk`, e está confirmado que nunca foram versionadas. Mas a pasta do repositório continua a ser o sítio errado: basta um `git add -f`, um zip para mandar a alguém, ou uma sincronização de backup.

```
mkdir -p ~/.ssh
mv "E:/Projects/WeeFly/Weefly Backend/concierge-weefly-key.pem" ~/.ssh/
mv "E:/Projects/WeeFly/Weefly Backend/concierge-weefly-key.ppk" ~/.ssh/
```

O OpenSSH do Windows recusa a chave se as permissões forem largas, com `UNPROTECTED PRIVATE KEY FILE`. Corrige antes de tentar ligar:

```
icacls "$USERPROFILE\.ssh\concierge-weefly-key.pem" /inheritance:r /grant:r "$USERNAME:R"
```

Entrar

O utilizador depende da imagem. Tenta por esta ordem até uma responder:

```
ssh -i ~/.ssh/concierge-weefly-key.pem ubuntu@52.30.78.0
ssh -i ~/.ssh/concierge-weefly-key.pem ec2-user@52.30.78.0
ssh -i ~/.ssh/concierge-weefly-key.pem admin@52.30.78.0
```

Se todas derem `Permission denied (publickey)`, o problema é o utilizador ou o whitelisting do IP, não a chave. Se der timeout, é o security group.

Reconhecimento

Antes de instalar o que quer que seja, saber o que lá está:

```
whoami && pwd
node -v                # tem de ser >= 18.17 - a Next 14 não arranca abaixo
npm -v
pm2 -v
nginx -v && sudo nginx -t
free -h                # a linha Mem, coluna total - ver Parte 3
df -h /                # espaço livre; o node_modules leva ~500 MB
systemctl list-timers | grep certbot
cat /etc/os-release | head -2
```

Aponta o valor de `free -h`. É o número que decide a Parte 3.

Ver também o que o NGINX já tem configurado, sem abrir ficheiros à sorte:

```
sudo nginx -T | grep -nE "server_name|client_max_body_size|proxy_pass|ssl_certificate "
```

Isto diz-te o ficheiro de configuração que o Certbot criou e se já existe algum `proxy_pass`. Guarda o caminho — vais precisar dele na Parte 6.

Estado verificável: tens sessão, sabes a RAM, sabes onde vive a configuração do NGINX.

Parte 2 - O código no servidor

A chave de deploy do GitHub

O repositório é privado. No servidor, gera uma chave dedicada e só de leitura — não uses o teu token pessoal, que dá acesso a tudo o que é teu:

```
ssh-keygen -t ed25519 -f ~/.ssh/github-deploy -N "" -C "ec2-concierge"
cat ~/.ssh/github-deploy.pub
```

Copia o resultado e vai a github.com/FabioRocha-pt/Weefly-Backend → **Settings** → **Deploy keys** → **Add deploy key**. Cola, dá-lhe um nome (`EC2 concierge`) e **deixa a caixa de escrita desmarcada**.

Depois, ensina o SSH a usá-la:

```
cat >> ~/.ssh/config <<'EOF'
Host github.com
  IdentityFile ~/.ssh/github-deploy
  IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
ssh -T git@github.com      # deve responder "successfully authenticated"
```

Clonar

```
sudo mkdir -p /var/www/concierge
sudo chown -R $USER:$USER /var/www/concierge
git clone git@github.com:FabioRocha-pt/Weefly-Backend.git /var/www/concierge
cd /var/www/concierge && git log --oneline -1
```

Estado verificável: o `git log` mostra o último commit do `main`.

Parte 3 · Memória — fazer isto antes do primeiro build

O `next build` chega a pedir 1,5 GB. Numa instância pequena é morto pelo kernel **sem mensagem nenhuma**: o build pára e não há erro para ler. É a avaria que mais tempo faz perder porque não se parece com falta de memória.

RAM (`FREE -H`)

O QUE FAZER

4 GB ou mais

Nada. Avança.

2 GB

Criar swap. É apertado sem ele.

1 GB

Criar swap, obrigatoriamente.

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h          # a linha Swap deixou de estar a zero
```

A alternativa, se o disco for pequeno, é construir noutra máquina e enviar a pasta `.next` já construída. Dá mais trabalho a cada deploy.

Parte 4 · As variáveis de ambiente

O ficheiro é `/var/www/concierge/.env.production`, na mesma pasta do `package.json`. O `next start` lê-o sozinho no arranque — o PM2 não precisa de `env_file`.

Os valores vêm do servidor antigo. Entra no `weefly.duckdns.org`, faz `cat` ao `.env.production` de lá, e traz os valores um a um. Não os copies do `.env.local` da tua máquina: esse tem

`NEXT_PUBLIC_SITE_URL=http://localhost:3000`, e esse valor errado manda cada cliente para um sítio que não existe — sem erro nenhum, porque os emails saem à mesma.

```
nano /var/www/concierge/.env.production
```

Obrigatórias — sem estas a aplicação não serve para nada

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=  
SUPABASE_SERVICE_ROLE_KEY=  
  
NEXT_PUBLIC_SITE_URL=https://concierge.weefly.africa
```

○ `NEXT_PUBLIC_SITE_URL` é o único valor que muda de conteúdo em relação ao servidor antigo. É ele que forma cada link de cliente e cada endereço dentro dos emails.

Email — sem estas o formulário responde bem e nada chega a ninguém

```
RESEND_API_KEY=  
CONCIERGE_FROM_EMAIL="WeeFly Concierge <concierge@weefly.africa>"  
CONCIERGE_TEAM_EMAIL=info@weefly.africa,info@weefly.cv  
RESEND_WEBHOOK_SECRET=
```

Operação

```
CONCIERGE_DIAGNOSE_TOKEN=  
PC_CRON_TOKEN=  
BO_ALLOWED_EMAILS=
```

○ `CONCIERGE_DIAGNOSE_TOKEN` é uma cadeia aleatória à tua escolha — é o que destranca a rota de diagnóstico da Parte 9. Sem ela, a rota responde 404 em produção.

○ `PC_CRON_TOKEN` é outra cadeia aleatória, e é obrigatória para a Parte 8. Gera as duas assim:

```
openssl rand -hex 24
```

○ `BO_ALLOWED_EMAILS` é apenas uma rede de segurança: a lista real de acessos ao back-office vive na tabela `bo_allowlist` da base de dados, e a variável é ignorada quando a tabela responde. Traz o valor do servidor antigo na mesma.

Opcionais — trazer do servidor antigo se lá estiverem

```
ANTHROPIC_API_KEY=  
CONCIERGE_NLP_MODEL=  
AMADEUS_CLIENT_ID=  
AMADEUS_CLIENT_SECRET=  
AMADEUS_API_BASE=  
AMADEUS_CURRENCY=  
WEEPAY_API_URL=  
WEEPAY_API_KEY=  
WEEPAY_WEBHOOK_SECRET=  
WEEPAY_DEFAULT_COUNTRY=  
WEEPAY_DEFAULT_METHOD=  
WHATSAPP_PHONE_NUMBER_ID=  
WHATSAPP_ACCESS_TOKEN=  
WHATSAPP_TEAM_NUMBER=  
WHATSAPP_API_VERSION=
```

As `BO_PASSWORD_*` que existem no teu `.env.local` **não vão para o servidor**. São usadas só pelo `scripts/seed-bo-users.mjs`, que corre da tua máquina.

Fechar o ficheiro

```
chmod 600 /var/www/concierge/.env.production
```

Tem lá dentro a chave de serviço do Supabase, que passa por cima de todas as políticas de RLS. Qualquer pessoa com esse valor lê e escreve tudo.

Estado verificável: `ls -l .env.production` mostra `-rw-----`.

Parte 5 - Construir e arrancar

```
cd /var/www/concierge  
npm ci # ci, não install: respeita o package-lock  
npm run build
```

Se o build parar sem erro, volta à Parte 3.

```
mkdir -p logs  
pm2 start ecosystem.config.js  
pm2 save  
pm2 startup # imprime um comando com sudo - copia-o e corre-o  
pm2 logs weefly-concierge --lines 40
```

O `ecosystem.config.js` já está no repositório e já está escrito para este servidor: uma instância, `127.0.0.1:3000`, `TZ=UTC`, reinício com espera crescente e tecto de memória a 700 MB.

O `pm2 startup` é o que faz a aplicação sobreviver a um reboot da máquina. Saltar esse passo só se nota no dia em que a AWS reinicia a instância.

Estado verificável:

```
curl -I http://127.0.0.1:3000      # HTTP/1.1 200 OK
pm2 status                       # weefly-concierge · online · 0 restarts
```

Parte 6 · O NGINX — o passo que parte em silêncio

Lê isto antes de mexer. O cliente carrega o comprovativo de pagamento no ecrã dele — JPG, PNG ou PDF, até 8 MB. O `client_max_body_size` do NGINX está por omissão em **1 MB**. Uma fotografia de um comprovativo tirada com um telemóvel moderno tem 3 a 6 MB.

O que acontece sem esta alteração: o NGINX corta o pedido com **413 antes de ele chegar à aplicação**. O cliente vê uma falha de upload que nenhum registo da aplicação explica, porque o pedido nunca lá chegou.

Abre o ficheiro que encontraste na Parte 1 e garante que o bloco `server` do 443 fica assim:

```
server {
    listen 443 ssl http2;
    server_name concierge.weefly.africa;

    # — ssl_certificate ... o que o Certbot já configurou, não mexer —

    # O comprovativo do cliente: 8 MB mais a folga do multipart.
    client_max_body_size 12M;
    client_body_timeout 120s;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;

        proxy_set_header Host                $host;
        proxy_set_header X-Real-IP           $remote_addr;
        proxy_set_header X-Forwarded-For    $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_read_timeout 90s;
        proxy_buffering off;
    }

    location /_next/static/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_cache_valid 200 365d;
        add_header Cache-Control "public, max-age=31536000, immutable";
    }
}
```

O `X-Forwarded-For` não é decorativo: o ecrã de consentimento promete guardar o IP de quem o deu, e a aplicação lê-o desse cabeçalho. Sem ele, todos os consentimentos ficam registados como vindos de `127.0.0.1`.

```
sudo nginx -t && sudo systemctl reload nginx
```

Não é preciso configuração de WebSocket. As actualizações em tempo real do back-office vão do browser directamente para o Supabase, não passam por aqui.

Estado verificável: `curl -I https://concierge.weefly.africa` responde 200.

Parte 7 · O que muda fora do servidor

Três sítios que não estão na máquina e que ficam a apontar para o endereço antigo se ninguém lá for.

ONDE	O QUE MUDAR
Painel do Resend	O webhook passa a <code>https://concierge.weefly.africa/api/webhooks/resend</code>
Supabase → Authentication → URL Configuration	Acrescentar <code>https://concierge.weefly.africa</code> ao Site URL e às Redirect URLs
Painel da WeePay, se já estiver ligado	O webhook passa a <code>https://concierge.weefly.africa/api/weepay/webhook</code>

O Supabase é o que parte o login do back-office: a aplicação constrói o retorno como `{origin}/auth/callback`, e o Supabase recusa redireccionar para um endereço que não esteja na lista.

Parte 8 · O cron que não existe neste servidor

No Vercel havia uma entrada em `vercel.json` a chamar `/api/pc/expire` de hora a hora. **Neste servidor não há nada equivalente, e é preciso criar.**

A rota fecha os pagamentos cujo prazo já passou. A expiração também acontece preguiçosamente sempre que alguém abre a página do cliente ou a ficha do back-office — o cron existe para o caso em que ninguém abre nem uma nem outra. Sem ele, um cliente que só volta ao link daqui a uma semana vê o ecrã de pagamento como se o preço ainda valesse.

O token não vai para dentro do `crontab`, que é legível por outros processos. Vai para um ficheiro só de root:

```

sudo mkdir -p /etc/weefly
echo "O_MESMO_VALOR_DE_PC_CRON_TOKEN" | sudo tee /etc/weefly/pc-cron-token
sudo chmod 600 /etc/weefly/pc-cron-token

sudo tee /usr/local/bin/weefly-pc-expire <<'EOF'
#!/bin/sh
TOKEN=$(cat /etc/weefly/pc-cron-token)
curl -fsS -H "Authorization: Bearer $TOKEN" \
  https://concierge.weefly.africa/api/pc/expire > /dev/null
EOF
sudo chmod 700 /usr/local/bin/weefly-pc-expire

sudo crontab -l 2>/dev/null | { cat; echo "0 * * * * /usr/local/bin/weefly-pc-expire"; } | s

```

Estado verificável: correr `sudo /usr/local/bin/weefly-pc-expire` à mão não imprime nada e sai com código 0. Se responder 404, o `PC_CRON_TOKEN` não está no `.env.production` ou a aplicação não foi reiniciada depois de o acrescentar.

Parte 9 · Os links antigos do duckdns

Há links `weefly.duckdns.org` já entregues a clientes — em emails enviados e em mensagens de WhatsApp. Se desligares a máquina antiga, esses links morrem.

O `duckdns` é teu, portanto tens uma saída limpa: aponta `weefly.duckdns.org` para `52.30.78.0` no painel do DuckDNS e deixa o servidor novo responder por ambos os nomes, redireccionando.

Depois de o DNS propagar:

```
sudo certbot --nginx -d weefly.duckdns.org
```

E um bloco que reencaminha tudo, preservando o caminho:

```

server {
    listen 443 ssl http2;
    server_name weefly.duckdns.org;

    # — ssl_certificate ... o que o Certbot acabou de configurar —

    return 301 https://concierge.weefly.africa$request_uri;
}

```

Um link antigo como `/pc/8IdIXN__M40YN1YoHjYmtTngIJaqj0rh` passa a chegar ao sítio certo, e passas a ter um servidor só.

Confirma que não há nada em falta antes de desligar a máquina antiga: no código não existe nenhum `duckdns` escrito à mão — está verificado. O domínio entra todo pelo `NEXT_PUBLIC_SITE_URL`.

Parte 10 · Antes de dizer que está no ar

Por esta ordem. Cada um apanha uma avaria diferente.

- **As migrações estão aplicadas.** São 19, até à `0019_sprint3_results.sql`.

Confirma no SQL Editor do Supabase. Sem elas a aplicação arranca e só falha quando alguém a usa.

- `https://concierge.weefly.africa/api/concierge/diagnose?token=...` responde e não traz `blockers`.

- **Submeter um pedido real em `/pc`.** Tem de chegar o email de confirmação ao cliente e o alerta à equipa.

- **Carregar um comprovativo de 5 MB.** É este o teste que apanha o `client_max_body_size`. Um ficheiro de 200 KB passa com a configuração errada e dá falsa confiança.

- **Abrir esse comprovativo no back-office.**
- **Emitir um caso de teste** e confirmar que o PDF aparece no link do cliente.
- **Na aba Comunicações do caso**, os avisos passam de `enviado` a `entregue`.

Se ficarem em `enviado`, falta o webhook do Resend da Parte 7.

- **Entrar no back-office** com uma conta real. Se o login rodar e voltar ao início, falta o Supabase da Parte 7.
- **Abrir um link antigo do duckdns** e confirmar que aterra no domínio novo.

Parte 11 · Segurança, depois de estar a funcionar

- **Substituir a chave SSH.** A chave que recebeste viajou por mensagem: está no

telemóvel de quem a enviou, no histórico da conversa e em todos os backups dessa conversa. Deixou de ser privada em qualquer sentido útil.

```
ssh-keygen -t ed25519 -f ~/.ssh/weefly-concierge -C "fabio@bonako"
ssh-copy-id -i ~/.ssh/weefly-concierge.pub <utilizador>@52.30.78.0
```

Testa a chave nova numa segunda sessão **antes** de fechar a primeira. Só depois de entrares com ela é que pedes ao Sarin para remover a original.

- **Porta 3000 fechada** no security group. Só o NGINX lhe fala, por localhost.
- **Portas abertas:** 443, 80 para redireccionamento e renovação do certificado, 22 restrita aos IPs da tabela do início.
- **Renovação do certificado:** `systemctl list-timers | grep certbot` tem de

mostrar um temporizador activo. Um certificado que expira ao domingo derruba o site e os links de todos os clientes.

- **Snapshot do EBS** depois de tudo configurado. A máquina não guarda dados — está tudo no Supabase — mas guarda a configuração, e é essa que custa uma tarde a refazer.

Deploys a partir daqui

```
cd /var/www/concierge
git pull
npm ci
npm run build
pm2 reload weefly-concierge
```

O `.env.production` não é tocado por nenhum destes comandos. Se acrescentares uma variável, tens de reiniciar: as variáveis são lidas no arranque.

Se correr mal

A máquina antiga fica de pé até tudo estar verificado. Voltar atrás é apontar o DNS de `concierge.weefly.africa` para o IP antigo e repor o `NEXT_PUBLIC_SITE_URL` que lá estava. Nenhum dado se perde no processo, porque nenhum dado vive nestas máquinas.

SINTOMA	CAUSA PROVÁVEL
<code>Permission denied (publickey)</code>	Utilizador SSH errado, ou o IP ainda não está no whitelisting
Ligação em timeout	Security group, não a chave
<code>UNPROTECTED PRIVATE KEY FILE</code>	Permissões do <code>.pem</code> no Windows — Parte 1
Build pára sem erro	Falta de memória — Parte 3
Upload de comprovativo falha, sem registo na aplicação	<code>client_max_body_size</code> — Parte 6
Formulário aceita e nenhum email sai	<code>RESEND_API_KEY</code> em falta; é deliberado que não dê erro
Links dos emails apontam para <code>localhost</code>	<code>NEXT_PUBLIC_SITE_URL</code> copiado do <code>.env.local</code>
Avisos ficam em <code>enviado</code> , nunca <code>entregue</code>	Webhook do Resend — Parte 7
Login do back-office roda e volta ao início	Redirect URLs no Supabase — Parte 7
<code>/api/concierge/diagnose</code> responde 404	<code>CONCIERGE_DIAGNOSE_TOKEN</code> em falta, ou aplicação não reiniciada
<code>/api/pc/expire</code> responde 404	<code>PC_CRON_TOKEN</code> em falta, ou aplicação não reiniciada
Consentimentos registados como <code>127.0.0.1</code>	<code>X-Forwarded-For</code> em falta no NGINX — Parte 6

SINTOMA

Site cai depois de um reboot da AWS

CAUSA PROVÁVEL

Faltou o `pm2 startup` — Parte 5